

Лабораторная работа №3 (локальный запуск). Открытые модели и MCP

Что сделаем

За эту работу вы соберёте маленького ИИ-агента: он отвечает на вопросы по документам компании и сам решает, когда искать в файлах. Всё работает на вашем ноутбуке, интернет нужен только для установки.

Шаг	Что делаем	Где	Баллы
1	Запускаем модель в Ollama	в аудитории	20
2	Запускаем готовый MCP-сервер	в аудитории	30
3	Дописываем одно место в агенте	в аудитории или дома	30
4	Проверяем агента и сравниваем две модели	дома	20

Код почти весь дан готовым. Ваша задача — запустить его, понять, как он работает, и дописать одно место в шаге 3.

Подготовка

Установите Python 3.10 или новее и Ollama с сайта ollama.com, затем выберите модель по объёму памяти ноутбука.

Память ноутбука	Ваша модель
16 ГБ и больше	qwen3.5:9b
8 ГБ	qwen3.5:4b
меньше 8 ГБ	qwen3.5:2b
ничего не запускается	работайте в паре с соседом

Объём памяти можно посмотреть так: в Windows — Диспетчер задач → Производительность → Память, в macOS — меню Apple → Об этом Mac.

Папка проекта. Создайте папку lab3, в ней папку docs, а в docs — два файла. Скопируйте в них текст ниже. Это учебные регламенты вымышленной компании.

Файл docs/hr_vacation.txt:

Политика отпусков ОсОО «Ала-Тех»
Ежегодный оплачиваемый отпуск составляет 28 календарных дней.
Заявление на отпуск подаётся не позднее чем за 14 дней до начала.
Отпуск можно разделить на части, одна из частей не менее 14 дней.

Оплата отпуска производится не позднее чем за 3 дня до его начала.
Вопросы по отпускам: отдел кадров, hr@ala-tech.kg
Файл docs/it_faq.txt:

FAQ IT-поддержки ОсОО «Ала-Тех»
Пароль от рабочей почты меняется каждые 90 дней.
Заявку на новый ноутбук согласует руководитель отдела.
VPN для удалённой работы выдаётся по заявке в IT-отдел.
Срок ответа на заявку в IT-поддержку – 1 рабочий день.
Телефон IT-поддержки: внутренний номер 555.
В итоге должно получиться так:

```
lab3/
├── docs/
│   ├── hr_vacation.txt
│   └── it_faq.txt
├── server.py
├── check.py
└── agent.py
```

Шаг 1. Запускаем модель (20 баллов)

Скачайте свою модель и задайте ей вопрос прямо в терминале. В примерах указана qwen3.5:4b, замените её на свою.

```
ollama pull qwen3.5:4b
ollama run qwen3.5:4b --verbose "Что такое ИИ-агент? Ответь в двух предложениях"
```

После ответа появится статистика. Найдите строку eval rate: это скорость модели в токенах в секунду.

Пока модель работает, откройте второй терминал и выполните ollama ps. Там видно, сколько памяти заняла модель.

Запишите в отчёт:

- какую модель вы запустили;
- скорость (eval rate);
- сколько памяти она заняла (ollama ps).

Шаг 2. MCP-сервер (30 баллов)

MCP-сервер даёт модели инструмент «поиск по документам». Код готовый: скопируйте его, запустите и разберитесь, что делает каждая часть.

2.1. Установите библиотеки. Команды выполняйте в папке lab3:

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install "mcp[cli]>=2,<3" ollama
```

2.2. Создайте файл server.py:

```
from pathlib import Path
from mcp.server.mcpserver import MCPServer
```

```
DOCS = Path(__file__).parent / "docs" # папка с документами
mcp = MCPServer("file-search") # создаём MCP-сервер

@mcp.tool() # эта функция станет инструментом для модели
def search_files(query: str) -> list[dict]:
    """Ищет слово или фразу во всех документах компании (папка docs).
    Используй, когда вопрос касается правил, сроков или контактов компании."""
    hits = []
    for f in DOCS.glob("*.txt"):
        for line in f.read_text(encoding="utf-8").splitlines():
            if query.lower() in line.lower():
                hits.append({"file": f.name, "text": line})
    return hits

if __name__ == "__main__":
    mcp.run()
```

2.3. Проверьте сервер без модели. Создайте файл `check.py` и запустите `python check.py`:

```
import asyncio
from mcp import Client, StdioServerParameters

server = StdioServerParameters(command="python", args=["server.py"])

async def main():
    async with Client(server) as client:
        tools = await client.list_tools()
        print("Инструменты:", [t.name for t in tools.tools])
        result = await client.call_tool("search_files", {"query": "отпуск"})
        for block in result.content:
            print(block.text)

asyncio.run(main())
```

Должны появиться инструмент `search_files` и шесть строк из `hr_vacation.txt` со словом «отпуск».

Ответьте в отчёте (2–3 предложения на вопрос):

1. Что делает строка `@mcp.tool()`?
2. Зачем нужен текст в тройных кавычках под `def search_files`? Кто его читает?
3. Поменяйте в `check.py` слово «отпуск» на «пароль». Что нашлось?

Шаг 3. Агент (30 баллов)

Агент соединяет модель и MCP-сервер: модель читает вопрос и сама решает, нужно ли искать в документах. Код почти готов, допишите одну строку на месте `TODO`.

Как работает агент:

1. Берёт у MCP-сервера описание инструмента `search_files` и показывает его модели.
2. Отправляет модели вопрос.
3. Если модель отвечает текстом — это готовый ответ.
4. Если модель просит инструмент — агент вызывает его через MCP, отдаёт результат модели и спрашивает снова.

Создайте файл `agent.py`. В строке `MODEL` укажите свою модель.

```
import asyncio
import ollama
from mcp import Client, StdioServerParameters

MODEL = "qwen3.5:4b" # ваша модель
server = StdioServerParameters(command="python", args=["server.py"])
SYSTEM = ("Ты помощник сотрудников компании. Отвечай только по документам. "
          "Если в документах нет ответа, так и скажи.")

async def ask(question):
    async with Client(server) as client:
        # 1. Берём у MCP-сервера описание инструментов и передаём модели
        tools = [{"type": "function",
                  "function": {"name": t.name,
                              "description": t.description,
                              "parameters": t.input_schema}}
                 for t in (await client.list_tools()).tools]
        messages = [{"role": "system", "content": SYSTEM},
                    {"role": "user", "content": question}]

        for step in range(5): # не больше 5 шагов
            # 2. Спрашиваем модель
            reply = ollama.chat(model=MODEL, messages=messages, tools=tools)
            messages.append(reply.message)

            # 3. Модель не просит инструмент -> это готовый ответ
            if not reply.message.tool_calls:
                return reply.message.content

            # 4. Модель просит инструмент -> вызываем его через MCP
            for call in reply.message.tool_calls:
                print(f"Шаг {step + 1}: {call.function.name}({call.function.arguments})")
                result = ... # TODO: вызовите инструмент через client
                text = "\n".join(c.text for c in result.content) or "ничего не
найдено"
                messages.append({"role": "tool", "tool_name": call.function.name,
                                "content": text})
            return "Не успел ответить за 5 шагов"

question = input("Ваш вопрос: ")
print(asyncio.run(ask(question)))
```

Подсказка к TODO: посмотрите, как инструмент вызывается в `check.py`. Здесь то же самое, только имя инструмента и аргументы берутся из `call.function.name` и `call.function.arguments`. Не забудьте `await`.

Запустите `python agent.py` и спросите: «Сколько дней отпуска положено сотруднику?». Агент должен напечатать строку Шаг 1: `search_files(...)` и ответить «28 календарных дней».

Маленькие модели иногда отвечают сразу, без поиска. Если так вышло, задайте вопрос ещё раз или переформулируйте его — это нормально.

Шаг 4. Проверка и сравнение моделей (20 баллов, дома)

Задайте агенту четыре вопроса на двух моделях и посмотрите, какая справляется лучше.

Две модели. Возьмите свою модель и соседнюю по размеру: qwen3.5:2b и qwen3.5:4b или qwen3.5:4b и qwen3.5:9b. Вторую модель скачайте через `ollama pull` и поменяйте её название в строке `MODEL`.

Заполните таблицу. «Искал?» — напечатал ли агент строку Шаг 1: `search_files`. «Ответ верный?» — совпадает ли ответ с документом.

Вопрос	Нужен поиск?	Модель 1: искал?	Модель 1: ответ верный?	Модель 2: искал?	Модель 2: ответ верный?
Сколько дней отпуска положено сотруднику?	да				
Как часто нужно менять пароль от почты?	да				
Сколько будет $2 + 2$?	нет				
Какая зарплата у директора?	да, но ответа нет				

В последнем вопросе правильное поведение — сказать, что в документах такой информации нет, а не придумать число.

Вывод (3–4 предложения): какая модель справилась лучше и какую вы бы выбрали для бота компании — быструю маленькую или медленную, но точную?

Отчёт и оценка

Отчёт сдаётся в OSC до начала следующего занятия, объём — 1–2 страницы.

Что должно быть в отчёте:

1. Модель, скорость и память из шага 1.
2. Скриншот вывода `check.py` и ответы на три вопроса из шага 2.
3. Ваша строка вместо `TODO` и скриншот работы агента из шага 3.
4. Таблица и вывод из шага 4.
5. Что не получилось и как вы это решили.
6. Какие ИИ-инструменты вы использовали и для чего.

Шаг	Баллы	Что проверяем
1. Модель	20	Модель запущена, есть скорость и память
2. MCP-сервер	30	check.py работает, есть ответы на 3 вопроса
3. Агент	30	TODO дописан, агент ищет в документах и отвечает
4. Сравнение	20	Таблица заполнена для двух моделей, есть вывод
Бонус	+10	Второй инструмент read_document(name), который читает файл целиком и не даёт выйти из папки docs (запрос ../server.py должен вернуть ошибку)

Размер модели на оценку не влияет. Если маленькая модель ошибается, опишите это в отчёте — это тоже результат.

Сроки: опоздание до 48 часов — минус 20%; позже чем через неделю — не принимается без согласования.

Если что-то не работает

Сначала найдите свою ошибку в таблице — большинство проблем решаются так.

Что видите	Что сделать
No module named 'mcp.server.fastmcp'	Вы взяли код из интернета. Используйте код из этого задания: <pre>from mcp.server.mcpserver import MCPServer</pre>
No module named 'mcp'	Не активировано виртуальное окружение. Выполните <code>source venv/bin/activate</code> (Windows: <code>venv\Scripts\activate</code>)
model does not support tools	Выбрана не та модель. Используйте модели qwen3.5 из таблицы в разделе «Подготовка»
check.py зависает	Уберите <code>print()</code> из <code>server.py</code> : сервер общается через тот же канал вывода
Ноутбук сильно тормозит	Модель слишком большая. Закройте браузер или возьмите модель на размер меньше
Агент не ищет, а отвечает сам	Для маленьких моделей это нормально. Задайте вопрос ещё раз или запишите это в таблицу шага 4